



Tecnológico de Monterrey

Actividad 7.1 (Waypoints)

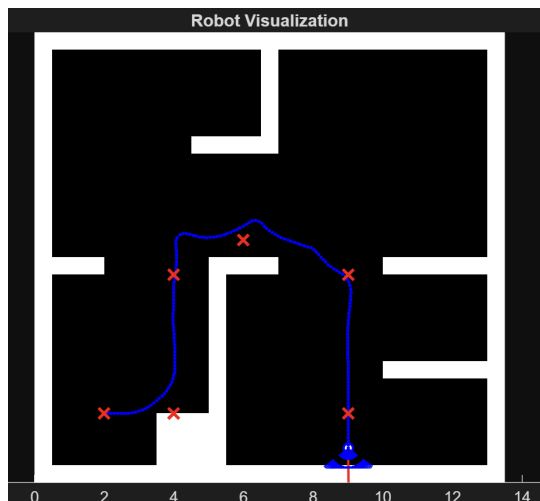
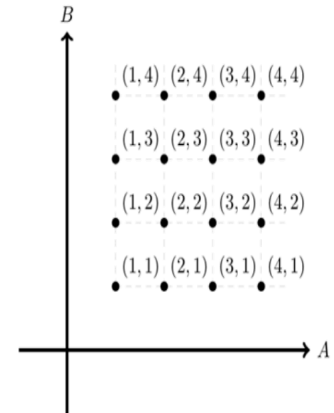
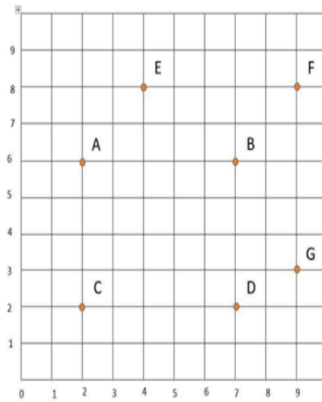
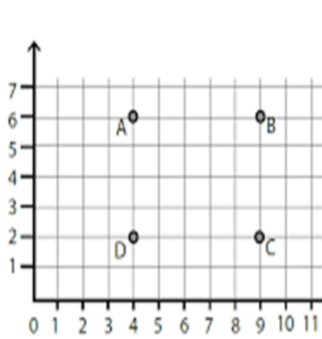
Ivan Ariel Rebollar Leon, A0173781

Escuela de Ingeniería y Ciencias, Instituto Tecnológico y de Estudios Superiores de Monterrey

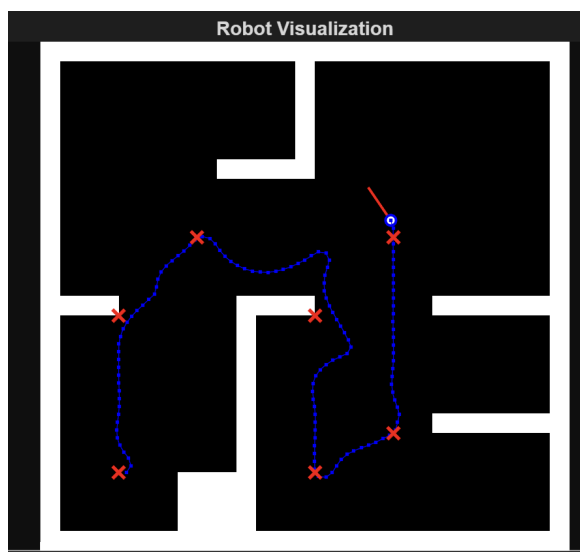
TE3002B.501: Implementación de robótica inteligente (Gpo 501)

20 de mayo, 2026

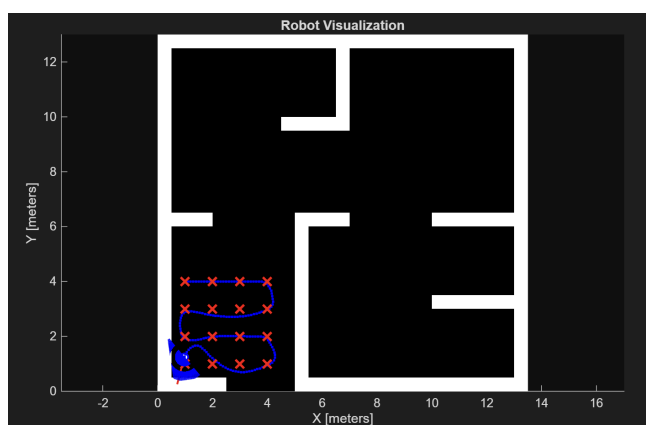
1. Crear un nuevo repositorio con el nombre: **Actividad 8.1 (SLAM de Lidar)**
2. Implementar el código requerido para generar el seguimiento de los siguientes waypoints de forma aleatoria, ajustando los parámetros: `sampleTime`, `tVec`, `initPose`, `lidar.scanAngles`, `lidar.maxRange`, `waypoints`, `controller.LookaheadDistance`, `controller.DesiredLinearVelocity` y `controller.MaxAngularVelocity`. Evadiendo los obstáculos del mapa de navegación "exampleMap" y "complexMap" (Para evadir los obstáculos es posible agregar way-points intermedios en el recorrido)



Para este primer escenario, la lista de waypoints se basó en el gráfico izquierdo (puntos A, B, C, D), introduciendo puntos intermedios para obligar al robot a mantenerse en el centro de los pasillos y no "cortar" por en medio de los muros. La `initPose` se fijó cerca del origen para enfilarse hacia el primer punto, mientras que el `sampleTime` y `tVec` se configuraron para un trayecto relativamente corto y fluido. Para la evasión, los `lidar.scanAngles` abarcaron un ángulo amplio y el `lidar.maxRange` se calibró para ver las paredes largas del túnel a tiempo.



Este entorno fue mucho más exigente porque los 7 objetivos (A hasta G) estaban distribuidos a lo largo de un laberinto complejo de múltiples habitaciones estrechas. Los waypoints tuvieron que trazar un camino sinuoso, y la `initPose` se estableció en un pasillo despejado, los sensores fueron vitales: el `lidar.maxRange` y los `lidar.scanAngles` actuaron en conjunto con el algoritmo VFH para detectar los marcos de las puertas antes de cruzarlas.

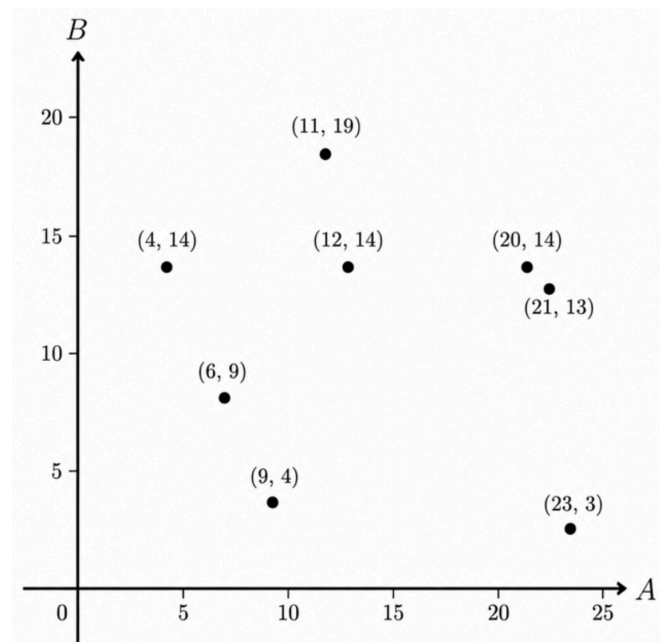


Los waypoints se extrajeron directamente de la matriz de 16 puntos, la `initPose` se forzó exactamente en la coordenada (1,1) para iniciar el patrón y aunque el área es pequeña, el `tVec` requirió holgura debido a las constantes maniobras calculadas en cada `sampleTime`, como los muros de la habitación estaban muy pegados a la ruta, los `lidar.scanAngles` y el `lidar.maxRange` le indicaban constantemente al robot sus límites, esta combinación obligó al robot a dar vueltas

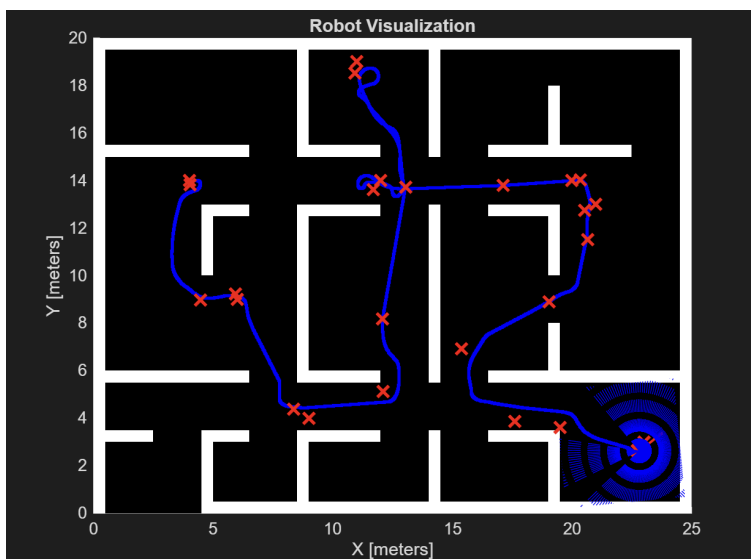
en U rotando casi sobre su propio eje, logrando pasar de una fila de puntos a la siguiente sin estamparse contra los bordes.

Actividad 8.2 (SLAM de Lidar)

1. Crear una carpeta nueva dentro del repositorio con el nombre: **Actividad 8.2 (SLAM de Lidar)**
2. Implementar el código requerido para generar el seguimiento de los siguientes waypoints de forma aleatoria, ajustando los parámetros: `sampleTime`, `tVec`, `initPose`, `lidar.scanAngles`, `lidar.maxRange`, `waypoints`, `controller.LookaheadDistance`, `controller.DesiredLinearVelocity` y `controller.MaxAngularVelocity`. Evadiendo los obstáculos del mapa de navegación "complexMap" (Para evadir los obstáculos es posible agregar way-points intermedios en el recorrido)



Solución:



Si el controlador de seguimiento (Pure Pursuit) intentará navegar directamente en línea recta entre los 8 puntos originales del gráfico, el robot chocaba constantemente con los muros del laberinto, para solucionar esto, el código genera de forma automática otros waypoints (utilizando un Planificador de Rutas Probabilístico (PRM) y estos sub-waypoints y gracias a esto el robot puede esquivar la infraestructura del mapa mientras avanza con precisión hacia las coordenadas reales que verdaderamente debe seguir.

1. Adecuación de la Escala del Mapa e Inicio

- Acción: Se escaló la matriz original de MATLAB al espacio real de trabajo de 25 metros de ancho por 20 metros de alto, usando una resolución de 2 celdas por metro.
- Impacto en desempeño: Una resolución equilibrada permite que los sensores detecten los pasillos del laberinto con precisión milimétrica sin saturar la memoria ni alentar el procesamiento de la computadora.

2. Sintonización Fina del Controlador Pure Pursuit

- Acción: Modificación de los parámetros dinámicos clave:
- LookaheadDistance = 0.50.
- DesiredLinearVelocity = 0.70.
- MaxAngularVelocity = 2.0.

3. Implementación del Filtro Reactivo de Seguridad (VFH)

- Acción: Configuración del Histograma de Campo Vectorial acoplado a un sensor Lidar con rango de 3.5 metros.
- Impacto en desempeño: Actúa como un escudo de última instancia, si el robot pasa cerca de una esquina o un muro debido a la inercia, el VFH reduce la velocidad lineal a un 35% y prioriza el giro para centrar al robot en el pasillo, evitando colisiones laterales.

Al fusionar la planificación global (los sub-waypoints del PRM) con el control local (Pure Pursuit + VFH), el script logra un comportamiento impecable, el robot no se atasca con errores de software, toma curvas fluidas a velocidades óptimas, esquiva las paredes con total seguridad y completa la validación de los 8 puntos del gráfico en un tiempo récord dentro de la simulación animada.